

you that it's more like an awesome sauce that makes everything taste better. Unit testing isn't designed to achieve some corporate quality initiative; it's not a tool for the end-users, or managers, or team leads. Unit testing is done by programmers, for programmers. It's here for our benefit alone, to make our lives easier.

Put simply, unit testing alone can mean the difference between your success and your failure. Consider the following short story.

1.1 Coding With Confidence

Once upon a time—maybe it was last Tuesday—there were two developers, Pat and Dale. They were both up against the same deadline, which was rapidly approaching. Pat was pumping out code pretty fast; developing class after class and method after method, stopping every so often to make sure that the code would compile.

Pat kept up this pace right until the night before the deadline, when it would be time to demonstrate all this code. Pat ran the top-level program, but didn't get any output at all. Nothing. Time to step through using the debugger. Hmm. That can't be right, thought Pat. There's no *way* that this variable could be zero by now. So Pat stepped back through the code, trying to track down the history of this elusive problem.

It was getting late now. That bug was found and fixed, but Pat found several more during the process. And still, there was no output at all. Pat couldn't understand why. It just didn't make any sense.

Dale, meanwhile, wasn't churning out code nearly as fast. Dale would write a new routine and a short test to go along with it. Nothing fancy, just a simple test to see if the routine just written actually did what it was supposed to do. It took a little longer to think of the test, and write it, but Dale refused to move on until the new routine could prove itself. Only then would Dale move up and write the next routine that called it, and so on.

Dale rarely used the debugger, if ever, and was somewhat puzzled at the picture of Pat, head in hands, muttering various evil-sounding curses at the computer with wide, bloodshot eyes staring at all those debugger windows.

The deadline came and went, and Pat didn't make it. Dale's code was integrated and ran almost perfectly. One little glitch came up, but it was pretty easy to see where the problem was. Dale fixed it in just a few minutes.

Now comes the punch line: Dale and Pat are the same age, and have roughly the same coding skills and mental prowess. The only difference is that Dale believes very strongly in unit testing, and tests every newly-crafted method before relying on it or using it from other code.

Pat does not. Pat "knows" that the code should work as written, and doesn't bother to try it until most of the code has been written. But by then it's too late, and it becomes very hard to try to locate the source of bugs, or even determine what's working and what's not.

1.2 What is Unit Testing?

A *unit test* is a piece of code written by a developer that exercises a very small, specific area of functionality of the code being tested. Usually a unit test exercises some particular method in a particular context. For example, you might add a large value to a sorted list, then confirm that this value appears at the end of the list. Or you might delete a pattern of characters from a string and then confirm that they are gone.

Unit tests are performed to prove that a piece of code does what the developer thinks it should do.

The question remains open as to whether that's the right thing to do according to the customer or end-user: that's what acceptance testing is for. We're not really concerned with formal validation and verification or correctness just yet. We're really not even interested in performance testing at this point. All we want to do is prove that code does what we intended, and so we want to test very small, very isolated pieces of functionality. By building up confidence that the individual pieces

work as expected, we can then proceed to assemble and test working systems.

After all, if we aren't sure the code is doing what we think, then any other forms of testing may just be a waste of time. You still need other forms of testing, and perhaps much more formal testing depending on your environment. But testing, as with charity, begins at home.

1.3 Why Should I Bother with Unit Testing?

Unit testing will make your life easier. It will make your designs better and drastically reduce the amount of time you spend debugging.

In our tale above, Pat got into trouble by assuming that lower-level code worked, and then went on to use that in higher-level code, which was in turn used by more code, and so on. Without legitimate confidence in any of the code, Pat was building a “house of cards” of assumptions—one little nudge at the bottom and the whole thing falls down.

When basic, low-level code isn't reliable, the requisite fixes don't stay at the low level. You fix the low level problem, but that impacts code at higher levels, which then need fixing, and so on. Fixes begin to ripple throughout the code, getting larger and more complicated as they go. The house of cards falls down, taking the project with it.

Pat keeps saying things like “that's impossible” or “I don't understand how that could happen.” If you find yourself thinking these sorts of thoughts, then that's usually a good indication that you don't have enough confidence in your code—you don't know for sure what's working and what's not.

In order to gain the kind of code confidence that Dale has, you'll need to ask the code itself what it is doing, and check that the result is what you expect it to be.

That simple idea describes the heart of unit testing: the single most effective technique to better coding.

Chapter 2

Your First Unit Tests

As we said in the introduction, a unit test is just a piece of code. It's a piece of code you write that happens to exercise another piece of code, and determines whether the other piece of code is behaving as expected or not.

How do you do that, exactly?

To check if code is behaving as you expect, you use an *assertion*, a simple method call that verifies that something is true. For instance, the method `assertTrue` checks that the given boolean condition is true, and fails the current test if it is not. It might be implemented like the following.

```
public void assertTrue(boolean condition) {  
    if (!condition) {  
        abort();  
    }  
}
```

AssertTrue.java

You could use this assert to check all sorts of things, including whether numbers are equal to each other:

```
int a = 2;  
// ...  
assertTrue(a == 2);  
// ...
```

If for some reason `a` does not equal 2 when the `assertTrue` is called, then the program will abort.

Since we check for equality a lot, it might be easier to have an assert just for numbers. To check that two integers are equal,

for instance, we could write a method that takes two integer parameters:

```
public void assertEquals(int a, int b) {
    assertTrue(a == b);
}
```

AssertTrue.java

Armed with just these two asserts, we can start writing some tests. We'll look at more asserts and describe the details of how you use asserts in unit test code in the next chapter. But first, let's consider what tests might be needed before we write any code at all.

2.1 Planning Tests

We'll start with a simple example, a single, static method designed to find the largest number in a list of numbers:

```
static int largest(int[] list);
```

In other words, given an array of numbers such as [7, 8, 9], this method should return 9. That's a reasonable first test. What other tests can you think of, off the top of your head? Take a minute and write down as many tests as you can think of for this simple method before you continue reading.

Think about this for a moment before reading on...



How many tests did you come up with?

It shouldn't matter what order the given list is in, so right off the bat you've got the following test ideas (which we've written as "what you pass in" → "what you expect").

- [7, 8, 9] → 9
- [8, 9, 7] → 9
- [9, 7, 8] → 9

What happens if there are duplicate largest numbers?

- [7, 9, 8, 9] → 9

Since these are `int` types, not objects, you probably don't care which 9 is returned, as long as one of them is.

What if there's only one number?

- `[1] → 1`

And what happens with negative numbers:

- `[-9, -8, -7] → -7`

It might look odd, but indeed -7 is larger than -9. Glad we straightened that out now, rather than in the debugger or in production code where it might not be so obvious.

To make all this more concrete, let's actually write a "largest" method and test it. Here's the code for our first implementation:

```

Line 1  public class Largest {
-
-      /**
-       * Return the largest element in a list.
-       *
-       * @param list A list of integers
-       * @return The largest number in the given list
-       */
-      public static int largest(int[] list) {
10         int index, max=Integer.MAX_VALUE;
-         for (index = 0; index < list.length-1; index++) {
-             if (list[index] > max) {
-                 max = list[index];
-             }
15     }
-     return max;
- }
- }

```

Largest.java

Now that we've got some ideas for tests, we'll look at writing these tests in Java, using the JUnit framework.

2.2 Testing a Simple Method

Normally you want to make the first test you write incredibly simple, because there is much to be tested the first time besides the code itself: all of that messy business of class names, library locations, and making sure it compiles. You want to get all of that taken care of and out of the way with the very first, simplest test; you won't have to worry about it

anymore after that, and you won't have to debug complex integration issues at the same time you're debugging a complex test!

First, let's just test the simple case of passing in a small array with a couple of unique numbers. Here's the complete source code for the test class. We'll explain all about test classes in the next chapter; for now, just concentrate on the assert statements:

```
import junit.framework.*;
public class TestLargest extends TestCase {
    public TestLargest(String name) {
        super(name);
    }
    public void testSimple() {
        assertEquals(9, Largest.largest(new int[] {7,9,8}));
    }
}
```

TestLargest.java

Java note: the odd-looking syntax to create an anonymous array is just for your authors' benefit, as we are lazy and do not like to type. If you prefer, the test could be written this way instead (although the previous syntax is idiomatic):

```
public void testSimple2() {
    int[] arr = new int[3];
    arr[0] = 7;
    arr[1] = 9;
    arr[2] = 8;
    assertEquals(9, Largest.largest(arr));
}
```

TestLargest.java

That's all it takes, and you have your first test. Run it and make sure it passes (see the sidebar on the next page).

Try running this example before reading on...



Having just run that code, you probably saw an error similar to the following:

```
There was 1 failure:
1) testSimple(TestLargest)junit.framework.AssertionFailedError:
   expected:<9> but was:<2147483647>
   at TestLargest.testSimple(TestLargest.java:10)
```

Whoops! That didn't go as expected. Why did it return such a huge number instead of our 9? Where could that very large

How To Run A JUnit Test

If JUnit is integrated into your IDE, then running a test may be as easy as pressing a button and selecting from a list of available test classes.

Otherwise, you can always execute a `TestRunner` manually. There are several flavors of test runners. To run a GUI version that lets you pick and choose classes (and which remembers them from session to session), run the following class:

```
java junit.swingui.TestRunner
```

You'll probably be able to run the `junit.swingui.TestRunner` class from your IDE. If not, run it from the command line using the `jre` or `java` command (as shown).

To run a test using the textual UI, as we've shown in this book, use:

```
java junit.textui.TestRunner classname ...
```

For instance, to run the unit tests on the preceding page we'd compile the `Largest` and the `TestLargest` programs, then run the `TestRunner` on `TestLargest`:

```
javac Largest.java TestLargest.java
java junit.textui.TestRunner TestLargest
```

number have come from? It almost looks like the largest number... oh, it's a small typo: `max=Integer.MAX_VALUE` on line 2 should have been `max=0`. We want to initialize `max` so that any other number instantly becomes the next `max`. Let's fix the code, recompile, and run the test again to make sure that it works.

Next we'll look at what happens when the largest number appears in different places in the list—first or last, and somewhere in the middle. Bugs most often show up at the “edges.” In this case, edges occur when when the largest number is at the start or end of the array that we pass in. We can lump all three of these asserts together in one test, but let's add the assert statements one at a time:

```
import junit.framework.*;
public class TestLargest extends TestCase {
    public TestLargest(String name) {
        super(name);
    }
    public void testSimple() {
        assertEquals(9, Largest.largest(new int[] {7,9,8}));
    }
    public void testOrder() {
        assertEquals(9, Largest.largest(new int[] {9,8,7}));
    }
}
```

TestLargest.java

Type in that test and run it. Hey, it works! Now try it with the 9 in the middle (just add an additional assertion to the existing `testOrder()` method):

```
public void testOrder() {
    assertEquals(9, Largest.largest(new int[] {9,8,7}));
    assertEquals(9, Largest.largest(new int[] {8,9,7}));
}
```

TestLargest.java

We're on a roll. One more, just for the sake of completeness, and we can move on to more interesting tests:

```
public void testOrder() {
    assertEquals(9, Largest.largest(new int[] {9,8,7}));
    assertEquals(9, Largest.largest(new int[] {8,9,7}));
    assertEquals(9, Largest.largest(new int[] {7,8,9}));
}
```

TestLargest.java

Try running this example before reading on...



There was 1 failure:

```
1) testOrder(TestLargest)junit.framework.AssertionFailedError:
   expected:<9> but was:<8>
   at TestLargest.testOrder(TestLargest.java:4)
```

Why did the test get an 8 as the largest number? It's almost as if the code ignored the last entry in the list. Sure enough, another simple typo: the for loop is terminating too early. This is an example of the infamous "off-by-one" error. Our code has:

```
for (index = 0; index < list.length-1; index++) {
```

But it should be one of:

```
for (index = 0; index <= list.length-1; index++) {
for (index = 0; index < list.length; index++) {
```

The second expression is idiomatic in languages descended from C (including Java and C#), and it's easier to read, so let's go with that one. Make the change and run the tests again.

Now you can check for duplicate largest values; type this in and run it (we'll only show the newly added methods from here on out):

```
public void testDups() {
    assertEquals(9, Largest.largest(new int[] {9,7,9,8}));
}
```

TestLargest.java

So far, so good. Now the test for just a single integer:

```
public void testOne() {
    assertEquals(1, Largest.largest(new int[] {1}));
}
```

TestLargest.java

Hey, it worked! You're on a roll now, surely all the bugs we planted in this example have been exorcised by now. Just one more check with negative values:

```
public void testNegative() {
    int [] negList = new int[] {-9, -8, -7};
    assertEquals(-7, Largest.largest(negList));
}
```

TestLargest.java

Try running this example before reading on...



```
There was 1 failure:
1) testNegative(TestLargest)junit.framework.AssertionFailedError:
    expected:<-7> but was:<0>
    at TestLargest.testNegative(TestLargest.java:3)
```

Whoops! Where did zero come from?

Looks like choosing 0 to initialize max was a bad idea; what we really wanted was MIN_VALUE, so as to be less than all negative numbers as well:

```
max = Integer.MIN_VALUE
```

Make that change and try it again—all of the existing tests should continue to pass, and now this one will as well.

Unfortunately, the initial specification for the method “largest” is incomplete, as it doesn't say what should happen if the array is empty. Let's say that it's an error, and add some code

at the top of the method that will throw a runtime-exception if the list length is zero:

```
public static int largest(int[] list) {
    int index, max=Integer.MAX_VALUE;
    if (list.length == 0) {
        throw new RuntimeException("Empty list");
    }
}
```

Largest.java

Notice that just by thinking of the tests, we've already realized we need a design change. That's not at all unusual, and in fact is something we want to capitalize on. So for the last test, we need to check that an exception is thrown when passing in an empty array. We'll talk about testing exceptions in depth on page 33, but for now just trust us:

```
public void testEmpty() {
    try {
        Largest.largest(new int[] {});
        fail("Should have thrown an exception");
    } catch (RuntimeException e) {
        assertTrue(true);
    }
}
```

TestLargest.java

Finally, a reminder: all code—test or production—should be clear and simple. Test code *especially* must be easy to understand, even at the expense of performance or verbosity.

2.3 More Tests

We started with a very simple method and came up with a couple of interesting tests that actually found some bugs. Note that we didn't go overboard and blindly try every possible number combination; we picked the interesting cases that might expose problems. But are these all the tests you can think of for this method?

What other tests might be appropriate?

Since we'll need to think up tests all of the time, maybe we need a way to think about code that will help us to come up with good tests regularly and reliably. We'll talk about that after the next chapter, but first, let's take a more in-depth look at using JUnit.